Student Enrollment System

Senior High School (Grades 11 and 12)

Object-Oriented Programming Concepts and File-by-File Code Guide

Java (Swing) + MySQL via JDBC

Companion document - not part of the project folder

What this document covers

1. How the system works (big picture)
2. The 5 basic OOP concepts, with where each appears in the code
3. The 4 pillars of OOP, with where each appears
4. Additional OOP and design concepts used
5. Function and purpose of every file
6. Database tables
7. Honest notes and limitations

1. The Big Picture

The system lets Senior High School students register, apply for enrollment, upload requirements and submit payments, while Admin, Enrollment Staff and Cashier users manage sections, applications, requirements, fees and payments. The code is split into layers so that each class has one job. Every request moves downward through the layers:

Layer	Classes	Job
1. Entry point	App	Starts the program and opens the login window.
2. Screens (GUI)	LoginFrame, RegistrationFrame, EnrollmentFormFrame, AdminDashboard, StaffDashboard, CashierDashboard, StudentDashboard	What the user sees and clicks. Collects input and shows results.
3. Action helpers	AdminSectionActions, StaffEnrollmentActions, StudentEnrollmentActions, StudentRequirementActions, StudentPaymentActions	Hold the longer feature logic of a dashboard so the dashboard class stays readable.
4. Services	AuthService, EnrollmentService	Business rules (hash passwords, validate grade level, register account + student).
5. Data access (DAO)	UserDAO, StudentDAO, EnrollmentDAO, RequirementDAO, FeeDAO, PaymentDAO, StudentPaymentDAO, ReportDAO, DeleteDAO	The only classes that talk SQL. One DAO per area of data.
6. Connection	DbConnection	Creates the JDBC connection to MySQL (port 3307, database enrollment_system).
7. Models	User, StudentUser, AdminUser, EnrollmentStaff	Plain objects that carry data (who is logged in).

Example: what happens when a user logs in

```
LoginFrame.login()            reads username + password from the text fields
   -> AuthService.login()     hashes the password with SHA-256
      -> UserDAO.login()      runs: SELECT * FROM users WHERE username=? AND password_hash=?
         -> DbConnection.getConnection()   opens the MySQL connection
      <- returns a User object (or null if the login is wrong)
LoginFrame reads user.getRole() and opens the matching dashboard
   (admin / staff / cashier / student)
```

2. The 5 Basic OOP Concepts

There is no single official list, so this guide uses the five building blocks that every Java class is made of: **Class, Object, Attribute (field), Method, and Constructor**. They are the foundation that the four pillars in Section 3 are built on.

1. Class

Meaning: A blueprint that describes what data something has and what it can do. Written once, used many times.

Where in this project: Every .java file is a class. `User` is the blueprint for a logged-in person; `UserDAO` is the blueprint for a worker that reads and writes the users table; `LoginFrame` is the blueprint for the login window.

```
public class User {          // blueprint: a person who logs in
    private int userId;
    private String username;
    ...
}
```

2. Object

Meaning: A real thing created from a class using the keyword new. Each object has its own copy of the data.

Where in this project: `UserDAO.login()` creates a `User` object from one database row. `App.main()` creates a `DbConnection` object and a `LoginFrame` object. Every DAO field such as `private DbConnection db = new DbConnection();` is an object held inside another object.

```
return new User(rs.getInt("user_id"), rs.getString("username"),
                rs.getString("email"), rs.getString("role"));   // UserDAO
```

3. Attribute (Field / Instance Variable)

Meaning: A variable that belongs to an object and stores its state (its data).

Where in this project: `User` stores userId, username, email, role. `DbConnection` stores url, username, password. `LoginFrame` stores usernameField, passwordField and authService. `EnrollmentDAO.SectionOption` stores sectionId, sectionName, capacity, currentCount.

```
public class DbConnection {
    String url = "jdbc:mysql://localhost:3307/enrollment_system";
    String username = "root";
    String password = "120604";
}
```

4. Method

Meaning: A named block of code that describes a behavior. It is what an object can do.

Where in this project: `getConnection()` opens a connection; `addStudent()` inserts a student; `hashPassword()` turns a password into SHA-256; `viewEnrollmentRecords()` shows records on screen. Methods that read a private field are called getters, such as `getRole()`.

```
public boolean addRequirement(String name) {
    ...
    int rowInserted = ps.executeUpdate();
    return rowInserted > 0;
}
```

5. Constructor

Meaning: A special method with the same name as the class and no return type. It runs automatically when new is used and sets up the object.

Where in this project: `User(int, String, String, String)` fills the four fields. `LoginFrame()` builds the whole window (title, size, buttons). `AdminSectionActions(AdminDashboard dashboard)` receives the dashboard it will work for.

```
public User(int userId, String username, String email, String role) {
    this.userId = userId;      // "this" = the object being built
    this.username = username;
    this.email = email;
    this.role = role;
}
```

3. The 4 Pillars of OOP

The four pillars are the main design ideas that make object-oriented code organized, safe and reusable. Below, each one is explained, then located in this project.

Pillar 1 - Encapsulation

Meaning: Keep data private inside the class and allow access only through methods, so other code cannot change data in a wrong way.

Where:

File	How encapsulation is applied
User	Fields userId, username, email, role are **private**. They can only be read through getUserId(), getUsername(), getEmail(), getRole(). There are no setters, so a User cannot be changed after login.
All DAO and service classes	The helper objects (private DbConnection db, private UserDAO userDAO, ...) are private. Screens cannot reach inside them.
PaymentDAO.readPayments()	A **private** method: only PaymentDAO itself may call it. Public methods getPayments(), getPendingPayments() and getPaymentsForStudent() are the allowed doors.
Dashboards	The text area, labels and DAO objects are private; the outside world can only use public methods such as getConnection() and showError().
DbConnection	The URL, username and password are kept inside one class so no other file hardcodes them. *(Its fields are package-private, not private, because the project requirements prohibit the private keyword in that class.)*

Pillar 2 - Inheritance

Meaning: A class can take over the fields and methods of another class using extends, then add or change things. This avoids repeating code.

```
public class StudentUser extends User { ... }      // StudentUser IS-A User
public class AdminUser extends User { ... }        // AdminUser   IS-A User
public class EnrollmentStaff extends User { ... }  // EnrollmentStaff IS-A User
public class LoginFrame extends JFrame { ... }     // LoginFrame IS-A window (JFrame)
```

Parent class	Child classes	What is inherited
User (own class)	StudentUser, AdminUser, EnrollmentStaff	userId, username, email, role and all four getters. Each child calls `super(...)` to pass data up to the parent constructor.
JFrame (Java Swing)	LoginFrame, RegistrationFrame, EnrollmentFormFrame, AdminDashboard, StaffDashboard, CashierDashboard, StudentDashboard	The ability to be a window: setTitle(), setSize(), setVisible(), dispose(), add(), setLayout(), and more - none of these were written by the team.

Pillar 3 - Polymorphism

Meaning: "Many forms." The same method name or type can behave differently depending on the actual object. Java has two kinds: method overriding (runtime) and method overloading (compile time).

Where	How it shows polymorphism
toString() overridden with @Override (AdminSectionActions.IdOption, EnrollmentDAO.SectionOption, StaffEnrollmentActions.EnrollmentOption, StudentDashboard.ChoiceItem)	Every Java object has a toString() method inherited from Object. These four classes **override** it to return a readable label such as "STEM-A (12/40 slots)". A JComboBox then calls toString() on each item and shows the right text for each different object.
User reference holding any role	LoginFrame stores the logged-in person in a variable of type `User` and passes it to whichever dashboard matches the role. Because StudentUser, AdminUser and EnrollmentStaff are Users, any of them could be placed in that same variable.
Dashboards as JFrame	LoginFrame calls `new AdminDashboard(user).setVisible(true)`. setVisible() is defined in the parent, but every dashboard (a different object) is displayed by the same call.
Lambdas and method references	addButton(panel, "View Fees", this::viewFees) passes behavior as a Runnable. The button runs a different action each time through the same call button.addActionListener(e -> action.run()).

Pillar 4 - Abstraction

Meaning: Hide the complicated details and show only what the caller needs. The caller asks "what", not "how".

Where	What is hidden from the caller
AuthService.login(username, password)	LoginFrame does not know about SHA-256 hashing, SQL or the users table. It just asks for a User.
DAO classes (UserDAO, StudentDAO, ...)	Screens call methods such as getStudentId(userId); the SQL, PreparedStatement, ResultSet and cleanup are hidden inside.
DbConnection.getConnection()	Callers do not know the URL, port, username or password. They only receive a Connection.
EnrollmentService.applyForEnrollment()	Hides the steps: check grade is 11 or 12, find the student id, then insert the application.
Action helper classes	A dashboard simply calls enrollmentActions.assignSection(); the long dialog and validation code lives elsewhere.

This project uses abstraction by hiding details inside classes and methods. It does not declare any `abstract` class or `interface` of its own - see Section 7.

4. Additional OOP and Design Concepts Used

Concept	Meaning	Where in this project
Composition ("has-a")	An object contains other objects as parts.	LoginFrame has an AuthService; AuthService has a UserDAO and a StudentDAO; every DAO has a DbConnection; StudentDashboard has three action helpers.
Association / delegation	One object passes work to another.	Action helpers receive their dashboard in the constructor (`new StaffEnrollmentActions(this)`) and call back into it.
this and super	this = the current object; super = the parent class.	`this.userId = userId` in User; `super(userId, username, email, "Admin")` in AdminUser; `this::viewFees` in dashboards.
Access modifiers	public, private, package-private control who can see a member.	public methods are the API; private fields and helpers are hidden (see Encapsulation).
Static nested classes	A small class defined inside another class and not tied to an outer object.	IdOption, SectionOption, EnrollmentOption, ChoiceItem (data holders for drop-down lists); PaymentDAO.FeeItem (private helper for payment line items).
Interfaces from Java libraries	A contract of methods a class promises to provide.	Lambdas implement ActionListener and Runnable; the code uses the JDBC interfaces Connection, PreparedStatement, Statement and ResultSet.
Overriding vs overloading	Overriding replaces a parent method; overloading reuses a name with different parameters.	Overriding is used (toString). Overloading is not a significant feature of this project, apart from the library methods it calls.
Generics and collections	Type-safe containers such as List<T>.	List<SectionOption> in EnrollmentDAO, List<Integer> in PaymentDAO, JComboBox<IdOption> in the admin screens.
Exception handling	try / catch / finally to react to errors safely.	Every database method uses try / catch (Exception e) / finally. The finally block closes ResultSet, PreparedStatement and Connection even when an error occurs.
DAO pattern	A class that isolates all database access for one subject.	UserDAO, StudentDAO, EnrollmentDAO, RequirementDAO, FeeDAO, PaymentDAO, StudentPaymentDAO, ReportDAO, DeleteDAO.
Layered architecture / Separation of concerns	Screen code, rules and database code live in separate classes.	GUI -> Actions -> Service -> DAO -> DbConnection (Section 1).
Single Responsibility Principle	A class should have one reason to change.	DbConnection only connects; ReportDAO only produces reports; AuthService only handles authentication and registration.
Transactions (data integrity)	Several SQL steps succeed together or not at all.	PaymentDAO.createPaymentRequest(), markPaymentPaid(), DeleteDAO.deletePayment() and EnrollmentDAO.assignSection() use setAutoCommit(false), commit() and rollback().
Security practices	Protect data from misuse.	Passwords are stored as SHA-256 hashes; PreparedStatement with ? placeholders prevents SQL injection.
Event-driven programming	Code runs in response to user events.	Buttons use addActionListener(...); App starts the UI with SwingUtilities.invokeLater(...).

5. Function and Purpose of Each File

Project root: `ENROLLMENT_SYSTEM_SHS/`. All Java source files are in `src/`.

5.1 Entry point and connection

File	Function and purpose
App.java	The program starts here (main method). It runs the database connection check, then opens the LoginFrame on the Swing event thread.
DbConnection.java	Holds the MySQL URL (port 3307, database enrollment_system), username and password. getConnection() returns a Connection, or null if the connection fails. connect() is a startup check that lists the users table to confirm the database works.

5.2 Models (data carriers)

File	Function and purpose
User.java	Represents the logged-in account: user id, username, email and role. Private fields with getters only. Parent class of the three role classes.
StudentUser.java	Child of User for students; passes the role label "Student/Parent" to the parent.
AdminUser.java	Child of User for administrators; passes the role label "Admin".
EnrollmentStaff.java	Child of User for enrollment staff; passes the role label "Staff".

5.3 Services (business rules)

File	Function and purpose
AuthService.java	hashPassword() creates the SHA-256 hash. login() hashes the typed password and asks UserDAO. register() validates required fields, creates the user account (role student) and then the student record.
EnrollmentService.java	applyForEnrollment() accepts only Grade 11 or 12, finds the student id, then asks EnrollmentDAO to save a Pending application. updateEnrollmentStatus() forwards a status change.

5.4 Data access objects (SQL lives only here)

File	Function and purpose
UserDAO.java	login() checks username and password hash and returns a User; addUser() inserts an account; getUserId() finds an id by username.
StudentDAO.java	addStudent() inserts a student profile; getStudentId() and getStudentName() look a student up by user id; findOrCreateStudent() updates or creates a student inside a transaction that another class controls.
EnrollmentDAO.java	apply() inserts an enrollment; getMyEnrollments() and getAllEnrollments() list applications; updateStatus() changes the status; getAvailableSections() lists sections with open slots; assignSection() puts a student into a section inside a transaction and rejects full or mismatched sections.
RequirementDAO.java	Manages the requirements list and submitted-requirement records: getRequirements(), addRequirement(), submitRequirement() (insert or update status), getSubmittedRequirements().
FeeDAO.java	addFee() inserts a fee for a strand and school year; getFees() lists fees; getTotalFees() sums the fees for one strand and year.
PaymentDAO.java	Staff and cashier side of payments: getDiscounts(); createPaymentRequest() builds the amount due from fees minus discounts and saves its detail lines in one transaction; getPayments(), getPendingPayments() and getPaymentsForStudent(); markPaymentPaid() verifies a payment.
StudentPaymentDAO.java	Student side of payments: list payment requests, show a request with its item breakdown, and addPaymentForStudent() submits a payment (status Pending) and marks the request Payment Pending.
ReportDAO.java	Report queries: enrollment totals by strand/grade/status, enrollees by strand, grade or section, incomplete requirements, and the sections report with enrolled counts.
DeleteDAO.java	All removal and cancellation actions: delete a requirement, cancel a payment, cancel an enrollment, deactivate a user, and delete a payment or payment request (with safeguards: paid records cannot be deleted).

5.5 Screens (GUI frames)

File	Function and purpose
LoginFrame.java	Login window. Validates the input, calls AuthService, then opens the dashboard that matches the role (admin, staff, cashier, student). Has a button to open registration.
RegistrationFrame.java	Student sign-up form (username, email, password, LRN, name, previous school). Checks required fields and calls AuthService.register().
EnrollmentFormFrame.java	Shows the student name and enrollment applications as a printable form with a Print button.
AdminDashboard.java	Admin menu: create, view, update and delete SHS sections; add and view school years, strands and fees; create staff/cashier accounts; enrollment report; logout.
StaffDashboard.java	Enrollment staff menu: view records, update status, assign section, delete an enrollment, review or delete requirements, view discounts, create payment requests, view payments, reports, guide, logout.
CashierDashboard.java	Cashier menu (payments only): view pending payments, view payment history, verify a payment (mark Paid), delete an eligible payment record, refresh the list, and logout.
StudentDashboard.java	Student menu: submit an application, view enrollments and requirements, print the enrollment form, upload or delete requirements, view payment requests and breakdown, submit payment, view payments. Also provides getConnection() for its helpers and the ChoiceItem list-item class.

5.6 Action helper classes (feature logic split out of dashboards)

File	Function and purpose
AdminSectionActions.java	Dialogs and database work for creating, updating and deleting (deactivating) sections, plus loadOptions() to fill strand and school-year drop-downs. Contains IdOption.
StaffEnrollmentActions.java	Update enrollment status, assign a section, pick an enrollment from a list, delete an enrollment record, and create a payment request. Contains EnrollmentOption.
StudentEnrollmentActions.java	Student application flow: loads school years and strands, shows the application form (LRN, name, previous school, year, strand, grade 11/12, enrollee type), validates it and saves the student profile and application; also prints the enrollment form.
StudentRequirementActions.java	Upload a requirement file (PDF/JPG/PNG, max 10 MB, copied to uploads/requirements) with a database record, and delete an eligible submission.
StudentPaymentActions.java	View payment requests and breakdown, submit a payment with method and reference, and view payment history.

5.7 Non-Java files and folders

File	Function and purpose
database/database_tables.sql	Creates the 14 tables: users, school_years, strands, students, sections, enrollments, student_requirements, requirements, submitted_requirements, fees, discounts, payment_requests, payment_request_details, payments.
lib/mysql-connector-j-26.7.0.jar	The MySQL JDBC driver that lets Java talk to MySQL.
lib/MANUAL/ (four .txt files)	README, user manual, code explanation and testing guide.
lib/GROUP 16	Group information, roles and setup notes. Its listed port (3306) differs from the code (3307).
bin/	Compiled .class files (one per class). Generated from src/ when the project is compiled.
.vscode/launch.json, settings.json	VS Code run configuration: main class App, classpath bin and the MySQL driver jar.

6. Database Tables

Table	Stores	Used mainly by
users	Accounts: username, password hash, email, role, active flag	UserDAO, DeleteDAO
students	Student profile linked to a user (LRN, names, previous school)	StudentDAO
school_years, strands	Reference data for school years and SHS strands	Admin screens, EnrollmentDAO
sections	Class sections with grade, strand, year, slot limit, active flag	AdminSectionActions, EnrollmentDAO
enrollments	An application: student, year, strand, section, grade, type, status	EnrollmentDAO, DeleteDAO
student_requirements	Uploaded requirement files and status	StudentRequirementActions, DeleteDAO, ReportDAO
requirements, submitted_requirements	Requirement list and submitted records	RequirementDAO
fees, discounts	Fee amounts per strand and year; discount rules	FeeDAO, PaymentDAO
payment_requests, payment_request_details	Amount due for an enrollment and its itemized lines	PaymentDAO, StudentPaymentDAO
payments	Payments submitted by students and verified by the cashier	PaymentDAO, StudentPaymentDAO

7. Honest Notes and Limitations

- **Unused subclasses.** StudentUser, AdminUser and EnrollmentStaff extend User, but no code creates them. UserDAO builds plain User objects and the role is kept as a text value. Inheritance is therefore demonstrated in the code structure, but role behavior is decided by comparing role text in LoginFrame, not by polymorphic subclasses.

- **Subclass constructors ignore the role argument.** Each child accepts a role parameter but passes a fixed label to super() instead.

- **No abstract classes or interfaces are declared.** Abstraction here comes from hiding details inside classes and methods.

- **No setters in User.** This is intentional encapsulation: the object is read-only after login.

- **DbConnection fields are not private.** This follows the project requirement that the class must not use private, static or final.

- **Documentation mismatch.** lib/GROUP 16 lists port 3306, while DbConnection uses port 3307 (the code is what runs).

- **Testing status.** The restructured project compiles. It was not run against a live MySQL database in the environment where this guide was prepared, so feature behavior should be confirmed on your own MySQL server.